Math 5315 – Numerical Methods for PDE– Fall 2025
w/Professor Du

Paul Carmody
Midterm Project – November 12, 2025

Apply the FTFS scheme to the Wave Equation with Periodic Boundary Conditions

FTFS over the Wave Function:

Given the Wave Equation with periodic boundary conditions over $[0, 1]$, that is

$$\begin{cases} u_t - u_x = 0 & x \in [0,1],\ t > 0 \\ u(x,0) = u(x,1) = 1. \end{cases} \quad (1)$$

The FTFS scheme works by evenly dividing the time and spatial dimensions separating points by Δt and Δx, not necessarily equal. Thus, each point over the domain can be calculated at $u_j^{k+1} = u_j^k + \Delta t$ and $u_{j+1}^k = u_j^k + \Delta x$. Then, by applying the forward time difference we approximate the derivative at each point.

$$\frac{\partial u}{\partial t} \approx \frac{u_j^{n+1} - u_j^n}{\Delta t} + O(\Delta t).$$

Then, applying the forward-space difference, the spatial derivative,

$$\frac{\partial u}{\partial x} \approx \frac{u_{j+1}^n - u_j^n}{\Delta x} + O(\Delta x)$$

.

Substituting these finite difference approximations into the wave equation (1):

$$\frac{u_j^{n+1} - u_j^n}{\Delta t} + \frac{u_{j+1}^n - u_j^n}{\Delta x} = 0$$

Solving for u_j^{n+1} in order to calculate successive points,

$$u_j^{n+1} = u_j^n - \frac{\Delta t}{\Delta x}(u_{j+1}^n - u_j^n)$$

We can then interate over both the time and spatial dimensions via Δt and Δx to provide successive data points. It will be convenient to introduce the Courant-Fredrichs-Lewey number, λ as

$$\lambda = \frac{\Delta t}{\Delta x}$$

and consequently

$$u_j^{n+1} = (1 + \lambda)u_j^n - \lambda u_{j+1}^n \quad (1)$$

Analytic vs Numerical solutions

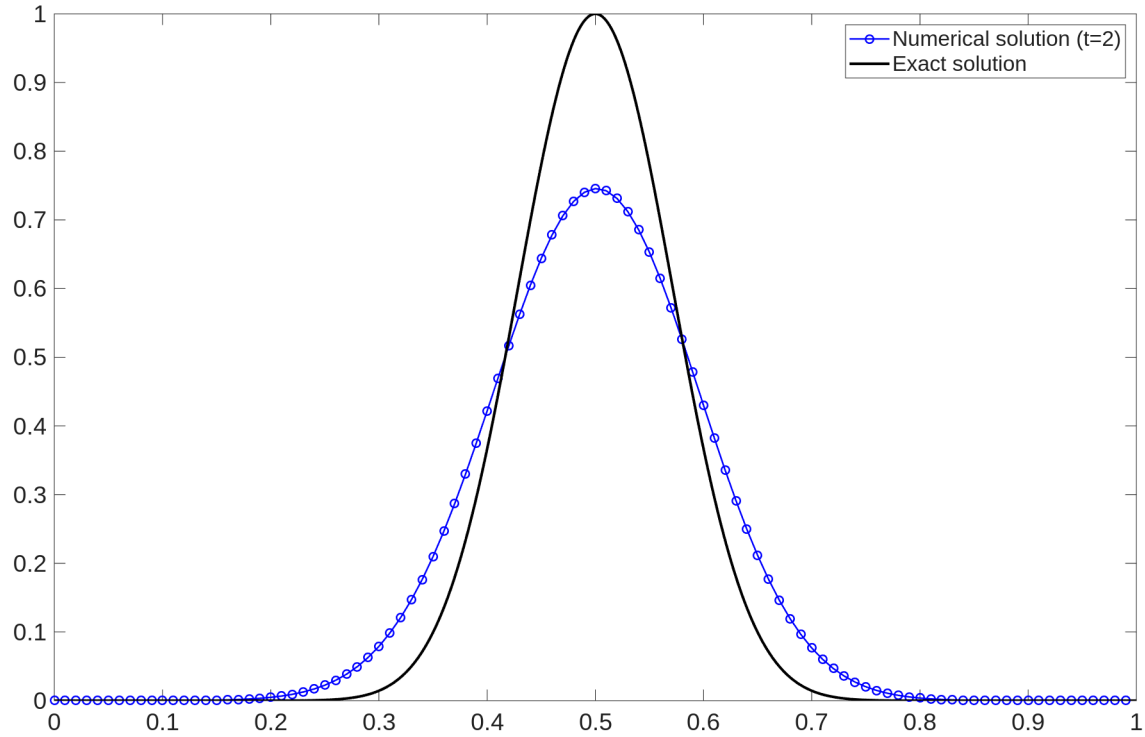

Figure 1: Analytic vs Numerical Solution

Here is the MatLab code that generated the graph.

```
% switch on frame saving
saveframes=0;

% solve the advection equation: du/dt + du/dx = 0.0
% using first order upwinding

% number of data points
M = 100;

% periodic interval dimensions (i.e. x \in [a,b))
a = 0; b =  1;

% data point spacing
% dx = (b-a)/M;
dx = 0.01;
M = (b-a)/dx;  % reset the number of data points

% coordinates of data points ("nodes")
x = linspace(a, b-dx, M);
u = zeros(1,M);

% start time
StartTime = 0;
EndTime = 2;
```

```
% dt chosen small to make space errors dominant
% dt = 0.55*dx;
dt = 0.008;

% compute number of time steps
Nsteps = ceil((EndTime-StartTime)/dt);

% modify dt to end exactly at EndTime
dt = (EndTime-StartTime)/Nsteps;

% initial condition
for i=1:1:M
  u(i) = exactsolution(x(i));
end
t = StartTime; % set time variable
fram = 1;
for n=1:Nsteps % begin time stepping

    um1 = [u(M),u(1:M-1)];          % u_{m-1}
    up1 = [u(2:M),u(1)];
    u1 = [u(2:M),u(1)];

     % update solution
%    unew = u-0.5*dt/dx*(up1-um1)+0.5*dt*dt*(up1+um1-2*u)/(dx*dx); % Wave Equation
%    unew = u-dt/dx*(u-um1);
   unew = u-dt/dx*(up1-u);  %  Transport Equation
   unew = u-dt/dx*(u-um1);

    u = unew; % finish one time step
    t  = t+dt;   % update time

    if(mod(n,20)==0 |n==Nsteps) % selective plotting

        plot(x, u, 'b-o', 'LineWidth', 1.2,'MarkerSize', 6); % plot numerical solution
        hold on;
        xplot = linspace(a, b, 1000); % plot exact solution at lots of points
        xmod = mod(xplot-t,b); % coordinate for exact pulse
        uexact = zeros(1,1000);
        for i=1:1:1000
          uexact(i) = exactsolution(xmod(i));
        end
        plot(xplot, uexact, 'k-', 'LineWidth', 2);hold off;
        %axis([a b -.5 1.5]);
        legend(sprintf('Numerical solution (t=%g)',t), 'Exact solution');
        set(gca,'fontsize',18);

        drawnow; pause(0.05);
        if(saveframes == 1)          % save frame to file
```

```
75                  if(fram<10)                fname = sprintf('anim_000%d.ppm',
                        fram);
76                  elseif(fram<100)     fname = sprintf('anim_00%d.ppm', fram);
77                  elseif(fram<1000)  fname = sprintf('anim_0%d.ppm', fram);
78                  end
79                  print('-dppm', fname);
80                  fram = fram+1;
81              end
82          end
83  end
84
85  clear
86  clc
```

Now let's draw attention to line 52 which encodes the FTFS expression. Further, the graph shows a dampened amplitude in the wave along with "softened" base, i.e., spreadout slightly near the horizontal axis. One might consider if the areas under each curve are the same.

Now see line 66 which calls "exactsolution" which is the analytic solution. Its code is here

```
1
2  function fn = exactsolution(x)
3
4  fn = sin(pi*x)^80;
5
6  %if(x>0.4 && x < 0.6)
7  %    fn = 1.0; %(abs(x-0.25) < 0.1);
8  %else if(x <= 0.55 && x >= 0.05)
9  %    fn = cos(2*pi*(x-0.3))^2;
10 %else
11 %    fn = 0.0;
12 %end
13 end
```

The commented out portions will be used in part (c) of the assignment.

Dispersion and Dissipation

Dispersion

We should consider our wave as made up of wave components, i.e., a series of pure wave expressions. The PDE dictates that all wave components should travel at the same speed. This, however, is where numerical approximation can deviate from the analytical expression. This deviation is called *dispersion* and is cumulative over the interval of interest. The effect is that the shape of the wave spreads out over time as seen in Figure 1.

The term 'wave components' indicates Fourier Analysis. In light of equation (1) we look towards a solution of the form

$$u_j^n = \hat{u}^n e^{ij\theta}$$

where j is the discrete spatial index with Δx giving $j\theta$. Applying this to (1) we obtain

$$\begin{aligned}
\hat{u}^{n+1} e^{ij\theta} &= (1+\lambda)\hat{u}^n e^{ij\theta} - \lambda \hat{u}^n e^{i(j+1)\theta} \\
\hat{u}^{n+1} &= (1+\lambda)\hat{u}^n - \lambda \hat{u}^n e^{i\theta} \\
\frac{\hat{u}^{n+1}}{\hat{u}^n} &= (1+\lambda) - \lambda e^{i\theta}
\end{aligned}$$

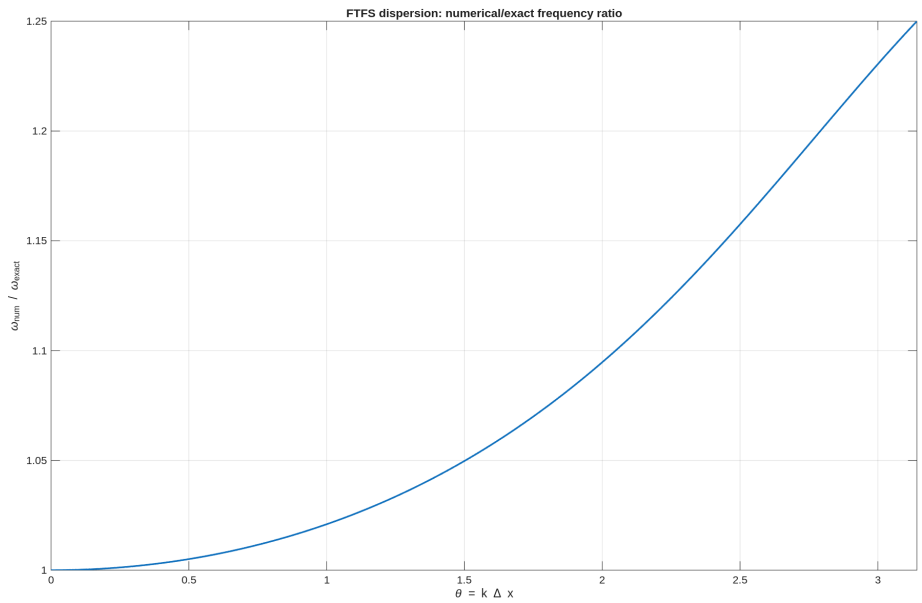

Figure 2: FTFS Dispersion Curve

With a small λ this demonstrates that each iteration has an increasing value over n (θ depends on n) and effected by the wave number. We'll define $G(\theta)$ as

$$G(\theta) = (1+\lambda) - \lambda e^{i\theta} = \frac{u^{n+1}}{u^n}$$

Dissipation

In addition to a dispersion of wave speeds over interations in numerical methods is the potential to effect the amplitude of the wave. We define this effect as *dissipation*. The ideal case, is that there is no dissipation then, of course, to minimize the effect that iterative solutions can introduce. Since it is an iterative solution, this, too, can be cumulative.

We can see that the ratio between successive iterations, $|G(\theta)|$, effects the progressive amplitude. That is, $|G(\theta)| = 1$ indicates no dissipation, $|G(\theta)| < 1$ implies a decaying amplitude and $|G(\theta)| > 1$ indicates an unstable scheme.

Indicators for Dispersion and Dissipation

Intuitively, the best indicator for dissipation would be to observe the effect of $G(\theta)$ over θ. This will indicate the effect of wave number on the dissipative effect. However, for dispersion an appropriate indicator would be to compare the appoximation of the wave component, $\omega(k)$, with the numerical approximation, $\bar{\omega}(k)$ where k is the wave number.

$$\omega(k) = -|G(\theta)|/\Delta t$$
$$\bar{\omega}(k) = -\theta/\Delta t$$

$\hat{\omega}$ is time dependent indicating that the dispersion will be increasing over successive iterations, progressively reducing accuracy. The code for Figure 2 and Figure 3 is

```
% DispDesp.m
% FTFS for u_t - u_x = 0 on [0,1] with periodic BC
% Delta x = 0.01, Delta t = 0.008 (lambda = dt/dx = 0.8)
% Produces: dissipation & dispersion curves, numerical vs
    analytical solutions,

```

```
clear; close all; clc;

%% Parameters
dx = 0.01;
dt = 0.008;
lambda = dt/dx;                % Courant number
L = 1;
Nx = round(L/dx);
x = (0:Nx-1)'*dx;              % column vector
T = 1.0;
nsteps = round(T/dt);          % should be integer (125)
if abs(nsteps*dt - T) > 1e-12
    error('T not an integer multiple of dt');
end

fprintf('Nx = %d, dx = %.4f, dt = %.4f, lambda = %.4f, nsteps = %d
    \n', Nx, dx, dt, lambda, nsteps);

%% FTFS operator (for Fourier analysis)
theta = linspace(0, pi, 2000);   % theta = k*dx (0..pi)
G = 1 - lambda + lambda*exp(1i*theta);  % amplification per time-
    step
amp = abs(G);                          % dissipation per step
phase = angle(G);                      % numerical phase per step
omega_num = -phase / dt;               % numerical angular freq (rad
    / unit time)
omega_exact = -theta / dx;             % exact angular freq (for wave
     u_t - u_x = 0 -> omega = -k)
ratio = ones(size(omega_num));
nz = abs(omega_exact) > 1e-16;
ratio(nz) = omega_num(nz) ./ omega_exact(nz);

%% Plot dissipation |G(theta)| vs theta
figure(1); clf;
plot(theta, amp, 'LineWidth', 1.6);
xlabel('\theta = k \Delta x');
ylabel('|G(\theta)| (amplification magnitude per step)');
title('FTFS dissipation curve: |G(\theta)| vs \theta');
grid on;
xlim([0 pi]);

%% Plot dispersion: omega_num/omega_exact vs theta
figure(2); clf;
plot(theta, ratio, 'LineWidth', 1.6);
xlabel('\theta = k \Delta x');
ylabel('\omega_{num} / \omega_{exact}');
title('FTFS dispersion: numerical/exact frequency ratio');
grid on;
xlim([0 pi]);
```

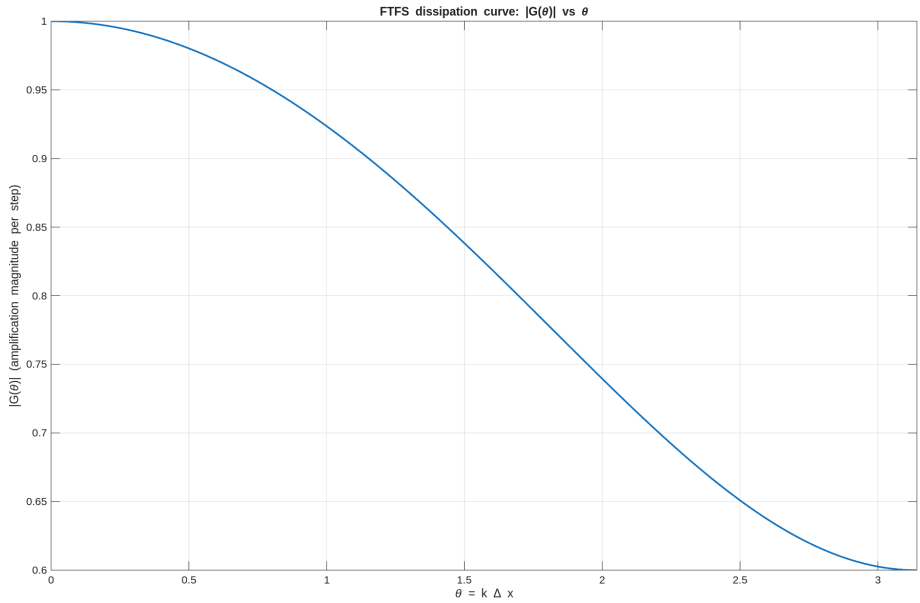

Figure 3: FTFS Dissipation Curve

This curve compares the dissipation to the wave number. A one indicating no dissipation. Thus the higher the wave number less accurate the numeric solution reflects the analytic solution.

Change the initial conditions

Part (b) of the assignment is: For the initial condition $u(x,0) = \sin^{40}(\pi x)$, plot the analytical and numerical solution at time $t = 1$, quantify the amount of dissipation.

As compared to Figure 1 we can see that the shape of the wave is much narrower for both the analytic solution and the numeric solution. And, by observation, the difference in maximum amplitude remains the same, indicating a similar dissipation factor.

Part (c) of the assignment is: Change the initial condition to $u(x,0) = 1$ for $0.4 \leq x \leq 0.6$ and $u(x,0) = 0$ otherwise. Plot the analytical and numerical solution at time $t = 1$, quantify the amount of dissipation. That is, the initial conditions $t = 0$ are

$$u(x,0) = \begin{cases} 1 & 0.4 \leq x \leq 0.6 \\ 0 & \text{otherwise} \end{cases}$$

Comparing the $\sin^{40}(x)$ example to the Square Pulse Example

The primary question is "How much of the wave amplitude was reduced by the numerical method?" This translates to a fraction dissipation, D.

$$D = \frac{|u(t)| - |\hat{u}(t)|}{|u(t)}$$

where u is the exact solution and $\hat{u}$ is the numeric solution.

Resulting in the following information:

	$E(0)$	$E(1)$	D
(b)	0.0889	0.0662	0.259
(c)	0.21	0.159	0.239

Where E is the comparative energy and D is the fractional dissipation. Thus the energy per amplitude drop off for (b) is 25% and for (c) it is 23.9%

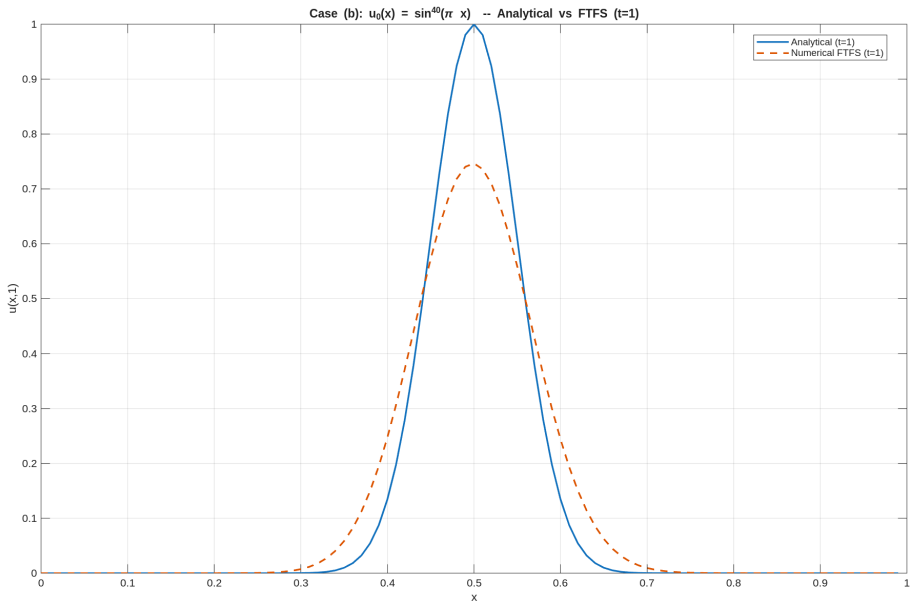

Figure 4: Changing the function from $\sin^{20}(\pi x)$ to $\sin^{40}(\pi x)$

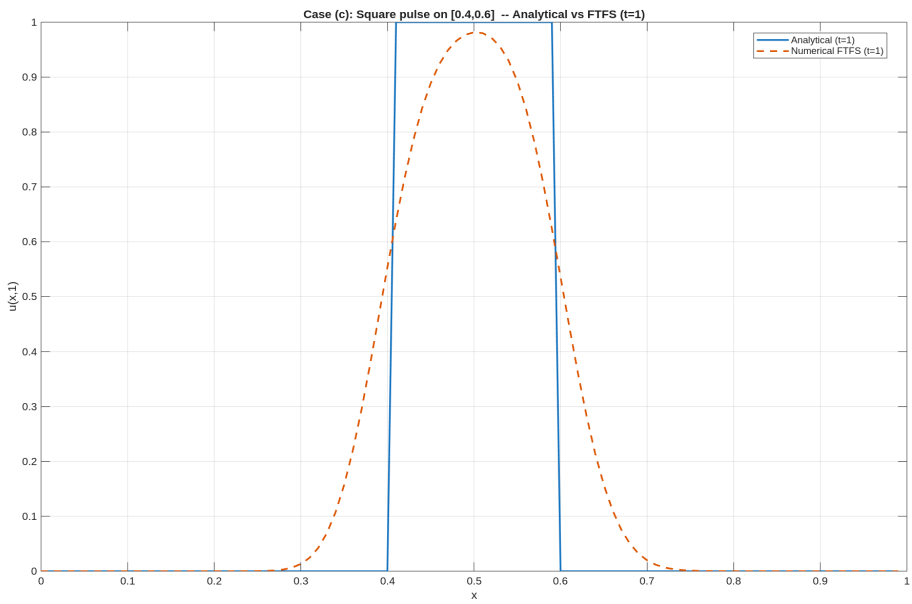

Figure 5: Changing the function from to a step function on $[0.4, 0.6]$

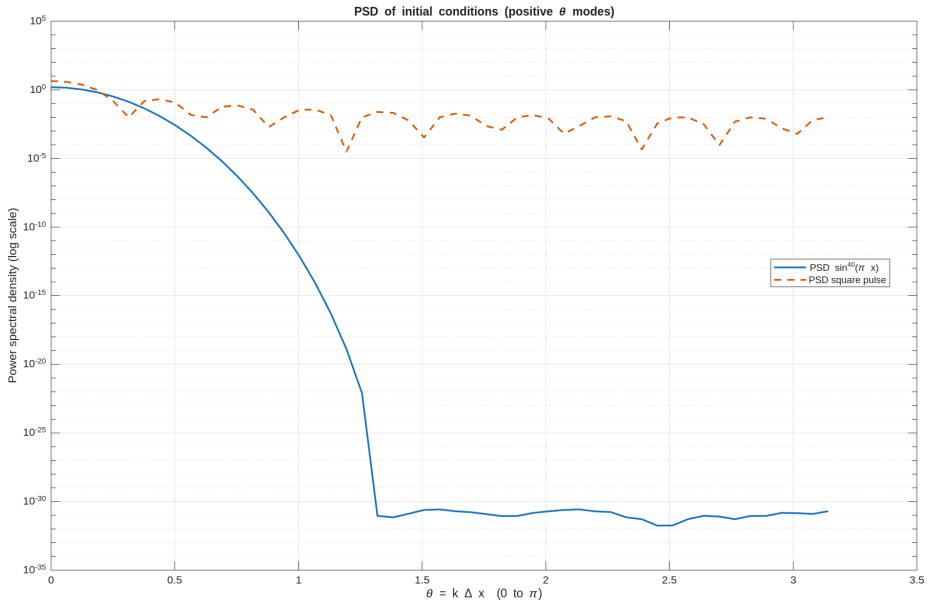

Figure 6: Power Spectrum Density of FTFS for $\sin^{40}(x)$ and a Square Pulse

Power Spectrum Density Analysis

Since the PDE requires a periodic solution, understanding the component waves is essential. Power Spectrum Analysis helps in this regard by indicating which wave forms dominate over the solution. The process is to take the vector numeric solutions and use the Fast Fourier Transform to generate a vector of complex values representing the wave numbers. We then compare the square of these values (being complex, the square provides us with real values for comparison). Figure 6 displays this comparison.

We can see that a Square Pulse requires many wave forms in order to approximate the solution (which is to be expected). In contrast, the sinusoidal wave uses only a few intitial wave numbers while the remaining drop off completely. Since different wave numbers travel at different speeds we can expect the Square Wave impulse have strong dispersion. Where as the sinusoidal solution will have fewer strong wave numbers and correspondingly less dispersion.

The code for this comparison and PSD analysis is

```
% Compare_b_c.m

clear; close all; clc;

%% Parameters
dx = 0.01;
dt = 0.008;
lambda = dt/dx;              % Courant number
L = 1;
Nx = round(L/dx);
x = (0:Nx-1)'*dx;            % column vector
T = 1.0;
nsteps = round(T/dt);        % should be integer (125)
if abs(nsteps*dt - T) > 1e-12
    error('T not an integer multiple of dt');
end

```

```
%% Helper: FTFS step function (periodic)
ftfs_step = @(u) (1-lambda)*u + lambda * circshift(u,-1);  % forward shift is j+1 -> circshift -1

%% ------- CASE (b): u0 = sin^{40}(pi x) -------
u0_b = sin(pi*x).^40;
u = u0_b;
for n = 1:nsteps
    u = ftfs_step(u);
end
u_num_b = u;
% Analytical solution: shift by +t (because u(x,t) = u0(x + t) for u_t - u_x = 0)
x_shift = mod(x + T, 1);
u_exact_b = sin(pi*x_shift).^40;

% Energy (L2) (using dx Riemann sum)
E0_b = sum((u0_b).^2) * dx;
En_b = sum((u_num_b).^2) * dx;
frac_diss_b = 1 - En_b / E0_b;

%% Plot analytical vs numerical for (b)
figure(3); clf;
plot(x, u_exact_b, '-', 'LineWidth', 1.6, 'DisplayName', 'Analytical (t=1)');
hold on;
plot(x, u_num_b, '--', 'LineWidth', 1.6, 'DisplayName', 'Numerical FTFS (t=1)');
xlabel('x'); ylabel('u(x,1)');
title('Case (b): u_0(x) = sin^{40}(\pi x)  -- Analytical vs FTFS (t=1)');
legend('Location','Best');
grid on;

fprintf('\nCase (b) (sin^{40}(pi x)):\n  E0 = %.6e, E_num(t=1) = %.6e, fractional dissipation = %.6e\n',...
    E0_b, En_b, frac_diss_b);

%% ------- CASE (c): square pulse on [0.4, 0.6] -------
u0_c = zeros(Nx,1);
% careful: include index where x in [0.4,0.6] inclusive
idx = (x >= 0.4) & (x <= 0.6);
u0_c(idx) = 1.0;

u = u0_c;
for n = 1:nsteps
    u = ftfs_step(u);
end
u_num_c = u;
% Analytical shift
u_exact_c = double((x_shift >= 0.4) & (x_shift <= 0.6));

% Energy
E0_c = sum((u0_c).^2) * dx;
```

```
En_c = sum((u_num_c).^2) * dx;
frac_diss_c = 1 - En_c / E0_c;

%% Plot analytical vs numerical for (c)
figure(4); clf;
plot(x, u_exact_c, '-', 'LineWidth', 1.6, 'DisplayName', 'Analytical (t=1)');
hold on;
plot(x, u_num_c, '--', 'LineWidth', 1.6, 'DisplayName', 'Numerical FTFS (t=1)');
xlabel('x'); ylabel('u(x,1)');
title('Case (c): Square pulse on [0.4,0.6] — Analytical vs FTFS (t=1)');
legend('Location','Best');
grid on;

fprintf('\nCase (c) (square pulse):\n  E0 = %.6e, E_num(t=1) = %.6e, fractional dissipation = %.6e\n',...
    E0_c, En_c, frac_diss_c);
```

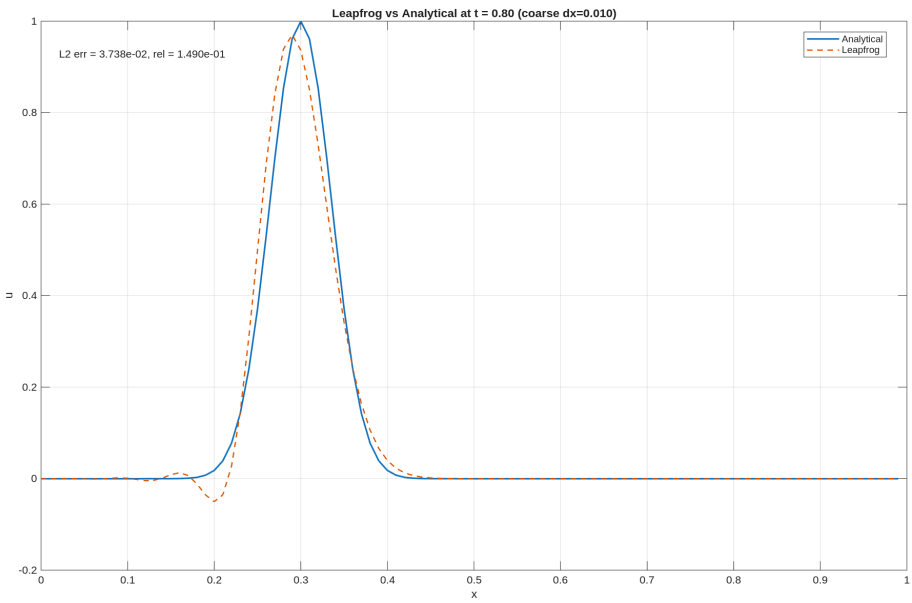

Figure 7: Leap Frog Graph for $\sin^{80}(x)$ at $t = 0.8$

Apply the Leap Frog scheme to the Transportation Equation with Periodic Boundary Conditions

Leap Frog:

We start in the same way as FTFS, that is, by dividing the time and space domains into subintervals. However, instead of using using the next element to determing both the partial derivatives, we use the element above and below. Thus, we get these two partial derivatives for the time and space dimensions.

$$\frac{\partial u}{\partial t} \approx \frac{u_j^{n+1} - u_j^{n-1}}{2\Delta t} \qquad \frac{\partial u}{\partial x} \approx \frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x}$$

Inserting these into the Transport Equation we get

$$\begin{aligned} \frac{u_j^{n+1} - u_j^{n-1}}{2\Delta t} &= \frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x} \\ u_j^{n+1} &= u_j^{n-1} - \lambda(u_{j+1}^n - u_{j-1}^n), \end{aligned}$$

where $\lambda = \Delta t/\Delta x$.

We analyze **disperstion and dissipation** again by the use of Fourier Transforms. Inserting $u_j^n = G^n e^{ij\theta}$ where $\theta = n\Delta x$ and we get

$$\begin{aligned} G^2 &= 2i\lambda \sin\theta G - 1 = 0 \\ G(\theta) &= \sqrt{1 - \lambda^2 \sin^2\theta} - i\lambda \sin\theta \\ |G(\theta)| &= 1 \end{aligned}$$

which indicates *no dissipation*. Dispersion, however, changes with respect to time (as outlined in great detail in "Finite Difference" by Le Veque, page 223).

Examining Figure 7 through Figure 9 we can see the effects of the dispersion over increasing values of t. Note that these graphs focus on low frequency wave numbers.

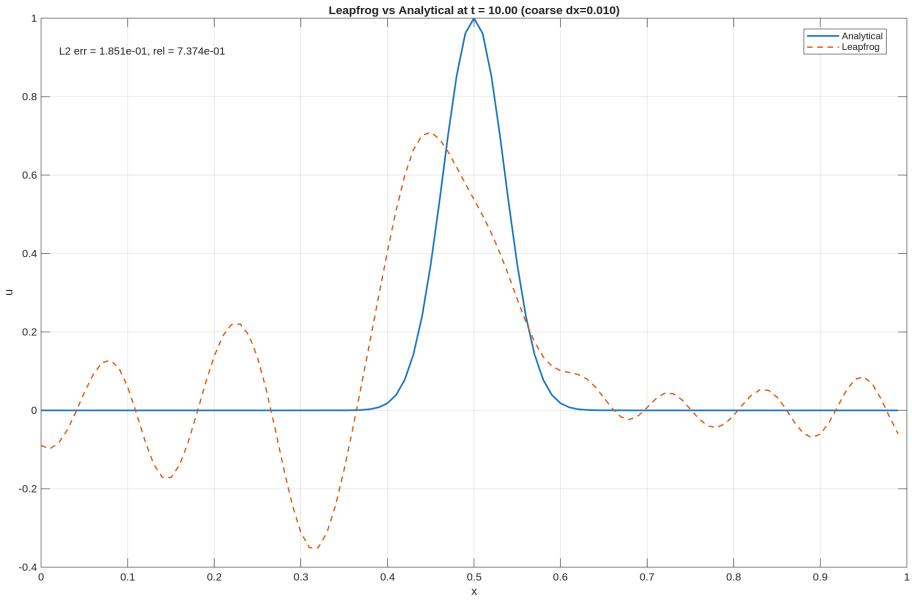

Figure 8: Leap Frog Graph for $\sin^{80}(x)$ at $t = 10$

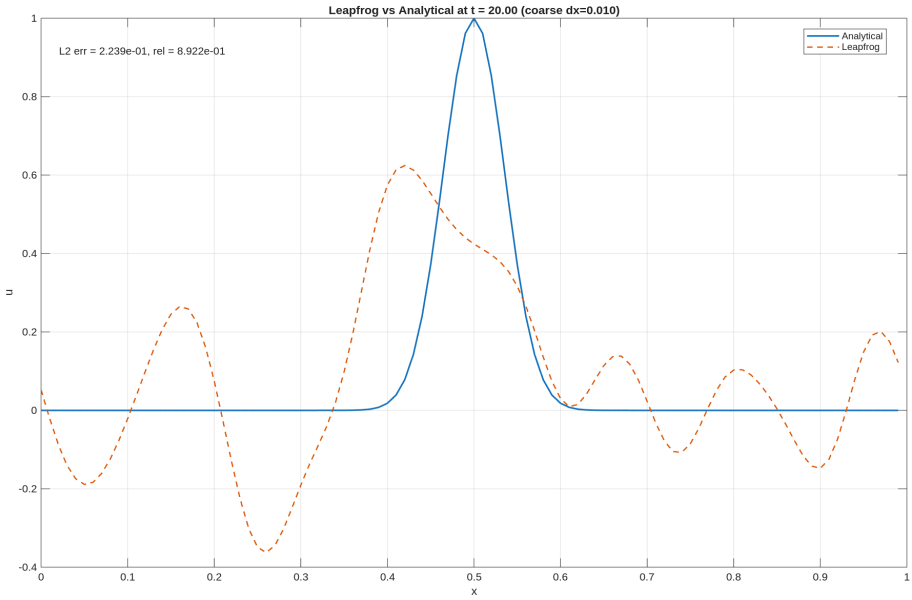

Figure 9: Leap Frog Graph for $\sin^{80}(x)$ at $t = 20$

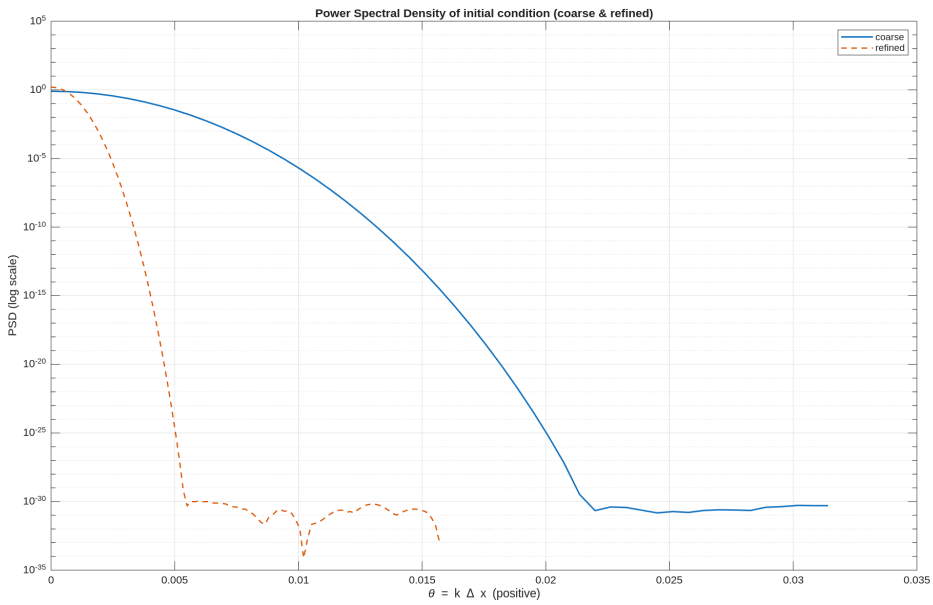

Figure 10: PSD on Leap Frog Scheme Anlaytic vs Numerical solutions

Power Spectrum Density Analysis

As described above in FTFS, Power Spectrum Density Analysis helps establish how accurate the numerical solution is in comparison with the analytic solution. We further anbalyze the 'coarse' frequency modes (long wavelengths and low frequency) with the 'refines' frequency modes (short wavelengths and high frequency). We can see from Figure 10 there is a distinct difference between these to categories of wave numbers noting that 'coarse'/lower frequences dominate the numeric solutions sigfnificatnly over the 'refined'/higher frequencies, supporting what is seen in Figure 7 through Figure 9.

The following is the MatLab code for all graphs and calculations.

```
% leapfrog.m
% Leapfrog for u_t + u_x = 0 on [0,1] periodic
% Initial: u0(x) = sin^{80}(pi x)
% dx = 0.01, dt = 0.008 (lambda = 0.8)

clear; close all; clc;

%% Parameters (coarse)
dx = 0.01;
dt = 0.008;
lambda = dt/dx;
L = 1.0;
Nx = round(L/dx);
x = (0:Nx-1)'*dx;          % column vector grid points (0..1-dx)
Tlist = [0.8, 10.0, 20.0];

fprintf('Coarse grid: Nx=%d, dx=%.4f, dt=%.4f, lambda=%.4f\n', Nx, dx,
    dt, lambda);

%% 1) Von Neumann analysis: dissipation & dispersion
theta = linspace(0, pi, 2000);              % theta = k*dx
a = lambda * sin(theta);
```

```
% Solve quadratic G^2 + 2i a G - 1 = 0 -> physical root:
G = sqrt(1 - a.^2) - 1i * a;                % select root with positive cos part
amp = abs(G);                               % |G(theta)|
% numerical frequency: phi = asin(a)  (omega_num * dt = phi)
phi = asin(a);                              % principal branch in [-pi/2, pi/2]
omega_num = phi / dt;                       % numerical angular frequency (rad/time)
omega_exact = (theta / dx);                 % exact for ut + ux = 0 is omega = k = theta/dx

% Avoid division by zero at theta=0
ratio = ones(size(omega_num));
idx = abs(omega_exact) > 0;
ratio(idx) = omega_num(idx) ./ omega_exact(idx);
ratio(~idx) = 1;

% Plot dissipation (should be 1)
figure(1); clf;
plot(theta, amp, 'LineWidth', 1.5);
xlabel('\theta = k \Delta x'); ylabel('|G(\theta)|');
title('Leapfrog dissipation curve (|G(\theta)|)'); grid on; xlim([0 pi]);

% Plot dispersion (omega_num / omega_exact)
figure(2); clf;
plot(theta, ratio, 'LineWidth', 1.5);
xlabel('\theta = k \Delta x'); ylabel('\omega_{num}/\omega_{exact}');
title('Leapfrog dispersion: \omega_{num} / \omega_{exact}'); grid on; xlim([0 pi]);

%% Utility functions: Leapfrog with Lax-Wendroff start, Lax-Wendroff full
function u_out = LF_evolve(u0, dx, dt, T)
    % Leapfrog with one Lax-Wendroff start step
    Nx = length(u0);
    lam = dt/dx;
    nsteps = round(T/dt);
    if nsteps == 0
        u_out = u0; return;
    end
    % compute u^1 via Lax-Wendroff
    u = u0;
    up = circshift(u, -1); um = circshift(u, 1);
    u1 = u - 0.5*lam*(up - um) + 0.5*(lam^2)*(up - 2*u + um);
    if nsteps == 1
        u_out = u1; return;
    end
    u_prev = u;        % u^{n-1} = u^0
    u_curr = u1;       % u^{n}   = u^1
    for n = 1:(nsteps-1)
```

```
        up = circshift(u_curr, -1);
        um = circshift(u_curr, 1);
        u_next = u_prev - lam*(up - um);    % u^{n+1} = u^{n-1} - lam(...)
        u_prev = u_curr;
        u_curr = u_next;
    end
    u_out = u_curr;
end

%% Initial condition
u0 = sin(pi*x).^80;

%% Run coarse Leapfrog at requested times
results_coarse = struct();
for k = 1:length(Tlist)
    T = Tlist(k);
    u_num = LF_evolve(u0, dx, dt, T);
    x_shift = mod(x - T, 1);                    % analytic shift (u(x,t)=u0(x - t) for ut+ux=0)
    u_exact = sin(pi*x_shift).^80;
    E_err = sqrt(sum((u_num - u_exact).^2) * dx);
    E_exact = sqrt(sum((u_exact.^2)) * dx);
    rel_err = E_err / E_exact;
    results_coarse(k).T = T;
    results_coarse(k).x = x;
    results_coarse(k).u_num = u_num;
    results_coarse(k).u_exact = u_exact;
    results_coarse(k).E_err = E_err;
    results_coarse(k).rel_err = rel_err;
    fprintf('Coarse LF: T=%.2f, L2 err = %.6e, rel = %.6e\n', T, E_err, rel_err);
end

%% Plot comparisons (coarse)
for k = 1:length(Tlist)
    T = Tlist(k);
    figure(10+k); clf;
    plot(results_coarse(k).x, results_coarse(k).u_exact, '-', 'LineWidth', 1.5); hold on;
    plot(results_coarse(k).x, results_coarse(k).u_num, '--', 'LineWidth', 1.2);
    title(sprintf('Leapfrog vs Analytical at t = %.2f (coarse dx=%.3f)', T, dx));
    xlabel('x'); ylabel('u'); legend('Analytical','Leapfrog','Location','Best'); grid on;
    text(0.02, 0.95, sprintf('L2 err = %.3e, rel = %.3e', results_coarse(k).E_err, results_coarse(k).rel_err), ...
         'Units','normalized','VerticalAlignment','top'); hold off;
end

%% PSD of initial condition (coarse and refined) - positive theta side
U0 = fft(u0);
```

```
dx_f = 0.005;
dt_f = 0.004;          % keep lambda = 0.8
Nx_f = round(L/dx_f);
x_f = (0:Nx_f-1)'*dx_f;
u0_f = sin(pi*x_f).^80;
psd0 = (abs(U0).^2)/Nx;
kfreqs = (0:Nx-1)'*(2*pi/Nx);      % discrete angular index (mapped to theta ~ k*dx)
npos = floor(Nx/2)+1;
theta_pos = kfreqs(1:npos) * dx;   % approximate theta = k*dx for plotting
figure(60); clf;
semilogy(theta_pos, psd0(1:npos), 'LineWidth', 1.4); hold on;
U0f = fft(u0_f); psdf = (abs(U0f).^2)/Nx_f;
kfreqs_f = (0:Nx_f-1)'*(2*pi/Nx_f);
theta_pos_f = kfreqs_f(1:floor(Nx_f/2)+1) * dx_f;
semilogy(theta_pos_f, psdf(1:floor(Nx_f/2)+1), '--', 'LineWidth', 1.2);
xlabel('\theta = k \Delta x (positive)'); ylabel('PSD (log scale)');
title('Power Spectral Density of initial condition (coarse & refined)');
legend('coarse','refined'); grid on; hold off;
```